context better. The following questions can be a starting point when looking for a contract testing tool:

- Can the tool support the programming language that your codebase is also using? We believe contract tests should be co-located in the consumer and provider codebase, so writing them in the same language increases their adoption.
- Can the tool support both the consumer and provider side adequately?
- Can the tool support different messages or communication types, for example, REST, GraphQL, and Kafka?
- Can the tool produce a contract that is generated from code instead of generating it outside the code?
- Are you looking at a more consumer-driven or provider-driven workflow? Some contract testing tools are more consumer-driven, while others are provider-driven.
- Do you want a contract testing tool that allows you to store the contracts on your own self-hosted infrastructure or to a fully managed cloud infrastructure?
- Are you looking for a tool that can also manage contract versions automatically?
- Can the tool integrate seamlessly in a CI pipeline?

By understanding the testing and technical requirements, you can then focus on how a tool can help you address their needs.

2.6.2 Identify your consumers and providers

Start by identifying who your consumers and providers are. If you're working on a team responsible for a public-facing website, then your team is a good candidate to be a consumer. From a consumer perspective, identify various APIs or data providers your website communicates with, and start with one of them. It's recommended to focus on one provider at a time and make sure that they are also willing to implement provider tests from their side because they need to verify the contract that you'll generate.